Digital Image Processing
Assignment 2

Released 8 October 2026 · **Due** 29 October 2026 ·

Report and code 50%, viva 50%. Your viva mark caps your report mark for each problem.
From scratch means NumPy array arithmetic. You may not call `scipy.ndimage`, `scipy.signal`, `cv2` or `skimage` for any operation a problem asks you to implement; use them to check your answers. `numpy.fft` is allowed everywhere – you are not asked to reimplement the FFT algorithm itself, only to use it correctly and understand what it computes.
Every number in your report must come from your machine. State your CPU, RAM and NumPy version.

#	Topic	Marks
1	DFT and the machinery of frequency	15
2	Frequency-domain filtering: ideal, Butterworth, Gaussian	15
3	Homomorphic filtering: illumination and reflectance	10
4	Edge-preserving smoothing: bilateral vs. Kuwahara	15
5	Denoising and the Wiener filter	15
6	Motion blur and deblurring	20
7	Lens distortion	10

Problem 1 DFT and the Machinery of Frequency (15 Points)

A standard 512×512 grayscale test image is provided. For Section 1.1, you must also synthesize your own 2-D image by summing at least three 2-D sinusoids with distinct user-chosen frequencies (u_i, v_i), amplitudes A_i, and phase shifts ϕ_i. Record your chosen parameters as ground truth; do not use external images for this specific parameter-recovery subtask.

1.1 Build and Validate the Discrete Fourier Transform (6 Points)

Implement the 1-D Discrete Fourier Transform (DFT) matrix W_N from scratch, and construct the 2-D DFT using row-column separability: $\mathbf{F} = \mathbf{W}_N \mathbf{f} \mathbf{W}_N^T$. Validate your implementation against `numpy.fft.fft2` by computing the maximum absolute error: $\max |F_{\text{custom}} - F_{\text{numpy}}|$.

a. **Conjugate Symmetry Proof:** Mathematically derive the conjugate symmetry property $F(u, v) = F^*(-u, -v)$ for real-valued spatial images $f(x, y)$. State precisely which spectral region is redundant and explain why standard visualizers display the full symmetric magnitude spectrum despite this redundancy.

b. **Empirical Verification of Theorems:** Empirically verify the *2-D Spatial Shift Theorem* and the *2-D Circular Convolution Theorem* using your custom implementation on the provided test image. Present explicit quantitative comparison tables of measured values versus theoretical predictions (plots alone are insufficient).



Figure 1: Provided test image.

c. **Sinusoid Parameter Recovery:** Compute the DFT of your synthetic sinusoid image. Extract and report the recovered frequencies, amplitudes, and phases from spectral peaks alone. Derive the theoretical spatial frequency resolution limit (Δf in cycles/image) imposed by grid dimension N, and explain its mathematical origin.

1.2 Phase vs. Magnitude Information Isolation (5 Points)

Pre-implementation Task: Before running any code, write an analytical prediction of the spatial reconstruction obtained by combining the *phase spectrum* of the test image with the *magnitude spectrum* of uniform white noise. Derive your prediction using the 2-D Shift Theorem and superposition principles rather than intuitive guessing.

a. Reconstruct the image using the phase of the test image combined with a *constant (flat) magnitude spectrum*. Display the output. Explain, using DFT properties, why flat magnitude isolates structural phase information more cleanly than swapping magnitudes between two natural images. Compare the result with your pre-implementation prediction and reconcile any discrepancies.

b. Reconstruct an image using the original magnitude spectrum of the test image while setting all phase values to zero ($\angle F(u, v) = 0$). Explain the structural characteristics of the resulting spatial image in terms of Fourier symmetry properties.

1.3 Padding, Wraparound, and Computational Overhead (4 Points)

Perform spatial filtering in the frequency domain by convolving an image with a small spatial kernel (e.g., 5×5 Laplacian or Sobel filter) via the 2-D DFT. Compare unpadded circular convolution with correctly zero-padded linear convolution.

a. Display visual evidence where circular wraparound corrupts edge boundary pixels. Mathematically derive the minimal zero-padding dimensions ($N_{x,\text{pad}}, N_{y,\text{pad}}$) required to guarantee exact equivalence between frequency-domain multiplication and spatial linear convolution for an $N \times N$ image and an $M \times M$ kernel.

b. Analyze the computational trade-off: Calculate the additional memory allocation factor and the increase in asymptotic runtime complexity $O(N \log N)$ incurred by zero-padding. Relate this

trade-off directly to internal padding mechanisms used in standard spatial convolution routines (e.g., `scipy.signal.fftconvolve`).

Deliverables for Section 1

- Quantitative error comparison table versus `numpy.fft.fft2`.
- Verification tables for the Shift and Convolution theorems.
- Dated, written analytical prediction submitted prior to running phase/magnitude experiments, accompanied by actual reconstruction plots and a reconciliation discussion.
- Recovered sinusoid parameters alongside the theoretical grid resolution limit derivation.
- Image outputs demonstrating circular wraparound corruption with complete padding size derivations.

Problem 2 Frequency-Domain Filtering: Ideal, Butterworth, and Gaussian (15 Points)

You are provided with four images:

- `characters_test_pattern.tif` (688×688), used for visual inspection of blurring and ringing;
- `text_gaps.tif`, low-resolution text with broken character strokes;
- `thumb_print.tif`, a fingerprint image for high-pass experiments;
- `cameraman.tif` (512×512), a natural image for the denoising study.

In addition, you must synthesize a 512×512 *step-edge image* $s(x, y)$ containing a bright square of intensity $B = 200$ and side length 256 centred on a background of intensity $A = 50$. Because its edges are perfectly straight and its jump height is known, this image is the ground truth for all quantitative ringing measurements in Section 2.2.

Unless stated otherwise, all filtering must follow the zero-padded pipeline you derived in Section 1.3: pad each $N \times N$ image to $P \times P$ with $P = 2N$, centre the spectrum, multiply by a real, zero-phase filter $H(u, v)$ defined on the $P \times P$ grid, invert, take the real part, and crop. Let $D(u, v) = \sqrt{(u - P/2)^2 + (v - P/2)^2}$ be the distance from the centre of the padded frequency rectangle. You may use `numpy.fft` for the transforms in this section, but every transfer function must be built by you from its formula; library filter routines are not permitted except where a subtask explicitly asks for a comparison.

2.1 Filter Construction and Transfer-Function Analysis (4 Points)

Implement a single function `make_filter(kind, P, D0, n=None, highpass=False)` that returns the $P \times P$ transfer function for

$$H_{\mathrm{ILPF}} = \begin{cases} 1 & D \le D_0 \\ 0 & D > D_0 \end{cases}, \qquad H_{\mathrm{BLPF}} = \frac{1}{1 + [D/D_0]^{2n}}, \qquad H_{\mathrm{GLPF}} = e^{-D^2/2D_0^2},$$

with high-pass versions defined by $H_{\mathrm{HP}} = 1 - H_{\mathrm{LP}}$.

a. **Cutoff Semantics:** Evaluate each low-pass filter analytically at $D = D_0$ and confirm numerically. Explain why using the *same* numerical D_0 for all three filters is not a like-for-like comparison. Derive the Gaussian parameter D_0' that places the Gaussian's half-amplitude point

($H = 0.5$) at the same radius as a Butterworth filter with cutoff D_0. Use this matched pairing in every later comparison and state it explicitly in your report.

b. **Energy-Based Cutoff Selection:** For the characters test pattern, compute the fraction of total spectral power enclosed within radius r,

$$\alpha(r) = 100 \cdot \frac{\sum_{D(u,v)\le r} |F(u,v)|^2}{\sum_{u,v} |F(u,v)|^2},$$

and report in a table the radii that retain $\alpha = 90\%, 95\%, 99\%$, and 99.5% of the power. Explain why such a large share of the power lies within a small radius, and why this does *not* mean the remaining high frequencies are visually unimportant.

c. **Inverse Design Problem:** A Butterworth low-pass filter of unknown order n and cutoff D_0 is measured to have $H = 16/17$ at $D = 10$ and $H = 1/17$ at $D = 40$. Recover n and D_0 analytically, showing every step. Then determine the smallest integer order n for which a Butterworth filter attenuates to at most 1% at $D = 2D_0$. Verify both answers with your `make_filter` implementation.

2.2 Ringing: Prediction, Measurement, and Reconciliation (6 Points)

Pre-implementation Task: Before running any code, write a dated analytical prediction of the horizontal intensity profile obtained by passing the step-edge image $s(x, y)$ through an ideal low-pass filter with cutoff D_0, measured along a row through the middle of the square and far from its corners. Your derivation must:

- use the fact that the Fourier transform of an image that varies only in x is confined to the line $v = 0$, so that only the one-dimensional slice $H(u, 0)$ of a radially symmetric filter affects the profile;
- express the predicted edge response in closed form using the sine integral $\mathrm{Si}(\cdot)$;
- predict (i) the percentage overshoot relative to the jump height $B - A$, and (ii) the distance in pixels from the edge to the first overshoot peak, as a function of D_0 and P.

Repeat the argument for the Gaussian low-pass filter and predict its overshoot.

a. **Ideal Filter Verification:** Filter $s(x, y)$ with ILPFs for $D_0 \in \{10, 20, 40, 80\}$. Define the overshoot as $\mathrm{OS} = 100 \cdot (\max g - B)/(B - A)$ measured on the profile row. Present a table comparing measured overshoot and first-peak distance against your predictions for each D_0. Reconcile any discrepancies, considering the discrete (rather than continuous) frequency grid. Separately, examine a profile that passes close to a corner of the square and explain why your one-dimensional argument no longer applies there.

b. **Butterworth Order Sweep:** Filter $s(x, y)$ with Butterworth low-pass filters at fixed $D_0 = 40$ for $n \in \{1, 2, 3, 5, 10, 20\}$. Tabulate the measured overshoot against n and compare it with the ideal-filter limit you predicted. In addition, compute the one-dimensional spatial-domain kernel $h(x) = \mathcal{F}^{-1}\{H(u, 0)\}$ for each order and report the ratio $|\min h|/\max h$. Using this ratio, explain why ringing is absent for $n = 1$ but present for higher orders, and prove that *any* filter whose spatial kernel is non-negative and sums to one cannot produce overshoot on a step edge.

c. **Visual Comparison:** Apply ILPF, BLPF ($n = 2$), and GLPF (using the matched D_0' from 2.1a) to the characters test pattern at the radii found in 2.1b. Display the results in a single grid with filter type along one axis and cutoff along the other. Identify, with arrows or zoomed

insets, at least two locations where ringing is visible in the ideal case, and comment on the sharpness–ringing trade-off across the three filter families.

d. **Gaussian Equivalence Check:** Derive the standard deviation σ_s (in pixels) of the spatial Gaussian kernel that corresponds to H_{GLPF} with parameter D_0 on a $P \times P$ grid. For $D_0 \in \{10, 40, 160\}$, compare your frequency-domain output against `scipy.ndimage.gaussian_filter` applied with your derived σ_s, and report $\max |g_{\text{freq}} - g_{\text{spatial}}|$ in a table. Explain why the error is not exactly zero and why it grows as D_0 becomes large.

2.3 Applications: Denoising, Repair, and Sharpening (5 Points)

a. **Noise–Blur Trade-off:** Add zero-mean Gaussian noise with standard deviation 20 (intensity range $[0, 255]$, fixed random seed reported) to `cameraman.tif`. For each of the three low-pass families, sweep D_0 from 5 to 150 and plot PSNR against D_0, with all three curves on a single plot. Report in a table the optimal D_0 and peak PSNR for each family. Explain, in terms of the transfer-function shapes from 2.1, why the curves peak where they do and why the ideal filter has the lowest peak.

b. **Character Repair:** Apply a Gaussian low-pass filter to `text_gaps.tif` and choose D_0 so that the broken strokes are joined while the characters remain legible. Display the before and after images, justify your chosen D_0, and explain what would happen if you replaced the Gaussian with an ideal filter at the same half-amplitude radius.

c. **High-Pass Filtering and the DC Term:** Apply IHPF, BHPF ($n = 4$), and GHPF to `thumb_print.tif` at a common half-amplitude radius of your choosing. Report the mean of each output and explain the result from the value of $H_{\text{HP}}(P/2, P/2)$. Then implement high-frequency emphasis, $H_{\text{HFE}} = k_1 + k_2 H_{\text{HP}}$, choose k_1 and k_2, and show that it restores the tonal information while sharpening ridges. Finally, threshold the BHPF output to produce a binary ridge map and comment on which filter gives the cleanest ridges and why.

Deliverables for Section 2

- The `make_filter` implementation, with radial-profile plots of all six transfer functions on a common axis.
- The cutoff-matching derivation, the power-retention table, and the full inverse-design solution with numerical verification.
- A dated, written ringing prediction submitted before running the step-edge experiments, along with the measured-versus-predicted overshoot tables, the Butterworth kernel-ratio table, the non-negative-kernel proof, and a reconciliation discussion.
- The σ_s derivation and the Gaussian equivalence error table.
- The PSNR sweep plot and optimum table, the repaired text image, and the high-pass and high-frequency-emphasis results, each with its accompanying explanation.

Problem 3 Homomorphic filtering: illumination and reflectance (10)

Data. An unevenly lit photograph (deep shadow and bright region in one frame – a doorway, a backlit face, an image with strong vignetting).

3.1 The multiplicative model (4)

State the model $I(x, y) = L(x, y)R(x, y)$ where L (illumination) is assumed slowly varying and R (reflectance) captures fine detail. Derive why taking the log converts this into an additive problem that a linear frequency-domain filter can act on.

- Explain precisely why the log must be taken *before* the Fourier transform, not after. What would go wrong if you filtered I directly in the log-magnitude spectrum instead of filtering $\log I$?

3.2 Implement and tune (6)

Implement the homomorphic high-frequency-emphasis filter: a Gaussian high-pass-like shape in the log domain of the form

$$H(u, v) = (\gamma_H - \gamma_L) \left[1 - e^{-c\, D^2(u,v)/D_0^2}\right] + \gamma_L,$$

with $\gamma_L < 1 < \gamma_H$, applied to $\log I$, then exponentiated back.

- Sweep γ_L, γ_H, D_0 as an image grid. Identify the settings that flatten illumination while preserving local contrast, and the settings that produce visible halos.
- Compare against plain global histogram equalisation on the same image (you may reuse your Assignment 1 implementation). Measure local contrast (e.g. local standard deviation in a shadow region) for both methods and state which better recovers shadow detail without blowing out the bright region, and why.

Deliverables. Derivation of the log-domain argument; parameter sweep grid with halo cases identified; local-contrast comparison table against histogram equalisation.

Problem 4 Edge-Preserving Smoothing: Bilateral vs. Kuwahara (15 Points)

Data & Input Requirements: A clean ground-truth image with distinct flat regions and sharp boundaries is provided. Add synthetic zero-mean additive Gaussian noise $\mathcal{N}(0, \sigma^2)$ to create your noisy input for Sections 2.1 and 2.2; report your selected σ. For Section 2.3, programmatically construct your own 2-D synthetic step-edge test images in NumPy (splitting an image into two distinct intensity regions I_1 and I_2 with additive Gaussian noise) across a controlled gradient of Noise-to-Contrast Ratios. Do not use a single fixed noisy image for the breakdown-ratio sweep.

4.1 Implementation and Matched-Strength Comparison (7 Points)

Implement both of the following edge-preserving spatial filters from scratch without using external image-processing library wrappers:

1. **Bilateral Filter:** Joint spatial Gaussian (σ_s) and intensity range Gaussian (σ_r) kernel with proper normalization.

1. **Classical Kuwahara Filter:** Four overlapping square sub-windows per pixel, outputting the mean of the sub-window exhibiting minimal intensity variance.

a. Formulate a clear mathematical definition for "matched effective smoothing strength" between the two filters (e.g., matching effective spatial footprint and variance reduction in flat regions). Justify your matching criteria.



Figure 2: Provided clean ground-truth image.

b. Evaluate filter performance against the clean ground truth using Peak Signal-to-Noise Ratio (PSNR) and Structural Similarity Index (SSIM). Include cropped zoomed-in comparisons illustrating region boundaries where edge preservation performance differs between the two methods.

4.2 Mitigating Kuwahara Blockiness and Complexity Analysis (5 Points)

Classical Kuwahara filtering exhibits distinct blocky, "painterly" artifacts at larger window sizes ($N \geq 11$) due to discrete sub-window selection.

a. **Mechanism Explanation:** Explain why the hard arg min variance operation specifically induces boundary blockiness rather than blur or ringing artifacts.

b. **Algorithmic Enhancement:** Implement *one* structural modification to mitigate blockiness (e.g., Generalised Kuwahara with continuous directional sub-windows, or continuous Soft-Kuwahara using softmax variance weighting). Present before-and-after cropped results at identical window sizes, and quantify the reduction in blockiness using an explicit metric (e.g., Total Variation or boundary gradient discontinuity).

c. **Per-Pixel Complexity Analysis:** Provide an algorithmic operational count (number of additions, multiplications, and exponentiations per pixel) comparing Classical Kuwahara, your Modified Kuwahara, and the Bilateral Filter. Identify which algorithm is computationally most expensive per pixel and explain why.

4.3 Quantitative Breakdown Analysis of Bilateral Filtering (3 Points)

The bilateral filter fails to preserve edges under severe noise conditions because its range kernel cannot differentiate true step edges from high noise variance.

a. Programmatically generate synthetic step-edge test cases (a 2-D array split evenly into two intensity levels I_1 and I_2 with step height $|\Delta I| = |I_1 - I_2|$ and added zero-mean Gaussian noise $\mathcal{N}(0, \sigma^2)$). Evaluate bilateral filtering across a range of Noise-to-Contrast Ratios:

$$\text{NCR} = \frac{\sigma_{\text{noise}}}{|\Delta I|} \tag{1}$$

Identify the precise critical threshold NCR^* where the filter transitions from edge preservation to smoothing across the boundary.

b. Plot an edge preservation metric (e.g., edge-region PSNR or percentage of boundary gradient magnitude retained) against NCR. Clearly annotate the empirical breakdown threshold NCR^* on the curve.

c. Provide a mathematical explanation of the breakdown in terms of the range Gaussian kernel $w_r(\Delta I) = \exp\left(-\frac{\Delta I^2}{2\sigma_r^2}\right)$ when noise variance approaches step height.

Deliverables for Section 4

- Matched-strength filter outputs accompanied by PSNR/SSIM evaluation tables and cropped comparison figures.
- Analytical explanation of Kuwahara blockiness, implementation code for the de-blocking modification, before/after visual metrics, and an algorithmic per-pixel complexity breakdown for all three filters.
- Synthetic step-edge generator code, quantitative breakdown curve for bilateral filtering with the critical ratio NCR^* identified, plotted, and mathematically explained.

Problem 5 Denoising and the Wiener filter (15)

Data. One clean, instructor-provided RGB image of at least 1024×1024 pixels for the controlled synthetic experiments. Use this same reference image in Problems 5 and 6. In addition, capture one genuinely noisy photo yourself for Problem 5.2 and one genuinely motion-blurred photo yourself for Problem 6.4.

5.1 The Wiener filter as an MMSE denoiser (7)

Assume that the clean image f and additive noise n are uncorrelated, wide-sense stationary random fields, with power spectral densities $S_{ff}(u, v)$ and $S_{nn}(u, v)$. For $g = f + n$, derive the frequency-domain linear MMSE estimate

$$\hat{F}(u, v) = W(u, v)G(u, v), \qquad W(u, v) = \frac{S_{ff}(u, v)}{S_{ff}(u, v) + S_{nn}(u, v)}.$$

Implement the Wiener denoiser. Normalize image intensities to $[0, 1]$. Estimate S_{nn} from the known injected noise variance or a flat region. Estimate S_{ff} from a smoothed or radially averaged noisy-image periodogram after subtracting the estimated noise power, or use a stated parametric $1/f^{\alpha}$ model. State your estimation method and its assumptions.

Add zero-mean white Gaussian noise at $\sigma \in \{5, 10, 20, 40\}$ 8-bit intensity levels, using the same noise realization for all methods at each σ. Do not clip the noisy input before filtering; clip only for display and metric calculation. Compare Wiener filtering with your Problem 2 Gaussian and Butterworth low-pass filters and your Problem 4 bilateral filter. Report PSNR and SSIM, the data range and SSIM implementation, and each method's parameter settings. If you use the clean reference to tune a parameter, label that result *oracle-tuned*.

Explain how Wiener shrinkage changes with frequency and noise power. Discuss how it differs from a fixed-cutoff low-pass filter. Do not assume that Wiener wins every comparison: its MMSE optimality applies to linear estimators under the stated statistical assumptions, and does not guarantee the best result against a nonlinear filter on every image or metric.

5.2 Test the assumptions (8)

- At $\sigma = 20$, repeat Wiener denoising with assumed noise powers of $0.5S_{nn}$ and $2S_{nn}$. Show these results beside the correctly specified result. Explain the residual noise or signal smoothing in terms of the Wiener formula, and report PSNR and SSIM for all three estimates.
- Explain why a correctly specified Wiener filter can still blur edges. Relate this limitation to linear estimation from stationary second-order statistics and explain what MMSE optimality does not guarantee about edges.
- Apply the white-noise Wiener filter to your own genuinely noisy photo, stating how you estimated its noise power. Compare it qualitatively with the synthetic cases using crops, residuals, or residual spectra. Discuss departures from white, stationary, signal-independent noise, such as shot noise, demosaicing, sharpening, and compression. Do not report PSNR or SSIM without a clean, registered reference.

Deliverables. Wiener, low-pass, and bilateral PSNR/SSIM table across σ; filter-parameter settings; figures showing noise-power mis-estimation; and a real-noise case study discussing the white-noise assumption.

Problem 6 Motion blur and deblurring (20)

Data. Use the clean instructor-provided image from Problem 5 for synthetic blur experiments. Also capture one genuinely motion-blurred photo yourself, using camera shake or a moving subject, for Problem 6.4.

6.1 Model and synthesize (5)

For uniform linear motion of length L pixels and direction θ, derive the continuous point spread function

$$h(x, y) = \frac{1}{L} \int_{-L/2}^{L/2} \delta(x - t\cos\theta,\ y - t\sin\theta)\, dt$$

and describe how you discretize and normalize it.

For the controlled experiment, state the boundary convention and use it consistently for both forward blurring and deconvolution. Circular convolution is recommended for the FFT experiment. If you use your Assignment 1 `conv2d_fft`, ensure its boundary handling matches the model used to construct H, or crop boundary-affected pixels during evaluation.

Blur the clean image and add zero-mean Gaussian noise at a stated SNR. Define SNR in your report; for example,

$$\mathrm{SNR}_{\mathrm{dB}} = 10 \log_{10} \frac{\sum_{x,y,c} f(x, y, c)^2}{\sum_{x,y,c} n(x, y, c)^2}.$$

Do not clip the noisy image before filtering.

Using spatial frequencies f_x, f_y in cycles per pixel and $\mathrm{sinc}(z) = \sin(\pi z)/(\pi z)$, derive the motion OTF

$$H(f_x, f_y) = \mathrm{sinc}(L(f_x \cos\theta + f_y \sin\theta))\,.$$

Show that its zero lines satisfy

$$f_x \cos\theta + f_y \sin\theta = \frac{m}{L}, \qquad m \in \mathbb{Z} \setminus \{0\}.$$

State that adjacent zeros are spaced $1/L$ along this frequency coordinate and that the zero lines themselves are perpendicular to the motion direction in the frequency plane. A discrete FFT grid may sample near, rather than exactly at, the theoretical zeros.

6.2 Naive inverse filtering and why it fails (5)

Implement $\hat{F} = G/H$ for the known synthetic blur. State any numerical floor used for division. Show the log-magnitude spectra of G and the raw inverse estimate before display clipping; mark the sampled frequencies nearest the predicted OTF zeros. Explain the noise amplification there and connect it to the zero spacing derived in Section 6.1.

State in one sentence why any nonzero noise can become arbitrarily large as $|H(u, v)|$ approaches zero.

6.3 Wiener deconvolution (5)

Derive and implement the scalar-K regularized Wiener deconvolution filter

$$\hat{F}(u, v) = \frac{H^*(u, v)G(u, v)}{|H(u, v)|^2 + K}.$$

State the image scaling and the interpretation of K as a constant noise-to-signal power ratio.

Sweep K logarithmically over several orders of magnitude. Show the low-K inverse-filter failure and high-K under-deconvolution. At each of at least three noise levels, report the K that maximizes PSNR against the clean reference and show how the optimum changes with noise level.

6.4 Estimate the blur, then remove it (5)

For your own motion-blurred photo, estimate the blur direction and approximate length from its log-magnitude spectrum. Display the spectrum and mark the estimated stripe spacing. Explain why the dark stripes are perpendicular to the motion direction. Build the estimated PSF, apply Wiener deconvolution, and report your estimated θ, L, and K.

Identify a visible artifact and connect it to a limitation of the linear, spatially invariant motion model, such as camera rotation, changing subject velocity, occlusion, or clipped highlights. Since the original sharp photo is unavailable, evaluate this result qualitatively rather than reporting ground-truth PSNR or SSIM.

Deliverables. PSF derivation and OTF zero-spacing formula; synthetic inverse-filter spectrum with predicted zero locations marked; Wiener K sweeps and optimum sensitivity to noise; and a real, self-captured blur case study with estimated θ, L, deblurred output, and a documented model failure.